Terminal Bench 4 (TB 4)

Task Proposal & Submission Guide for Trainers / Annotators

If you have built tasks for TB1 or TB2 before, most of your instincts still apply — you are still writing hard, self-contained terminal tasks with a clean solution and a strict verifier. What has changed is everything around your task. The whole workflow now lives on one platform. You submit your proposal there and get a reviewed verdict there; when the proposal is accepted a grant is recorded against it, and that grant is what lets you upload a task ZIP. Your upload is then admitted through Stage 1 sanity checks (gates G01–G06 plus the publication barrier (P04)), and only a task that clears Stage 1 goes on to Stage 2 model rollouts, where the difficulty gates are decided.

There is no Discord step and no canary string anywhere in this workflow. If you worked on TB4 before the platform, delete that habit: do not add canary comment lines to your files, and do not look for a canary_string field at upload. Nothing issues one and nothing asks for one.

0. The TB4 document set

This is part of a three-document set. Keep all three handy:

Document	What it covers
1. Proposal & Submission Guide (this document)	The end-to-end workflow: proposal, review verdict, building the task, Stage 1 admission, Stage 2 rollouts, and final QC sign-off.
2. TB4 — Static Checks Reference	Every automated static check and exactly what it enforces. Run these locally before uploading.
3. TB4 — Rubric Review Reference	Every rubric criterion the reviewer (LLM + human) grades your task against.

1. What changed from TB1 / TB2 → TB4

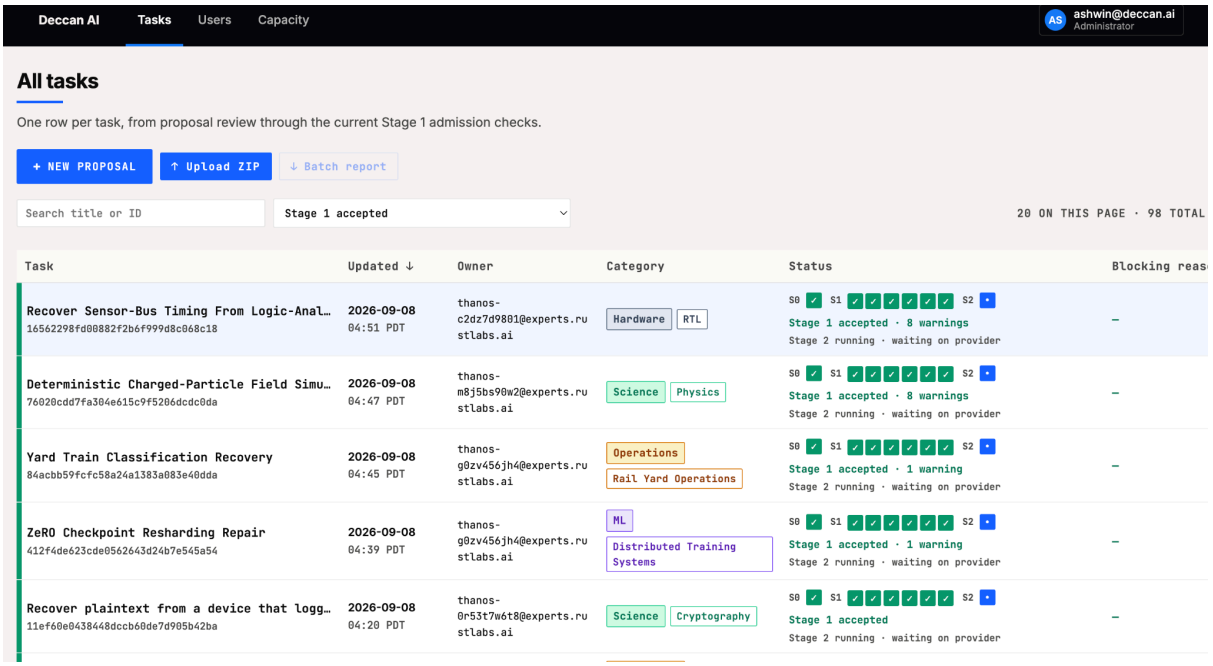
If you only remember a few things, remember these:

Area	TB1 / TB2 (old way)	TB4 (new way)
Getting started	Pick an idea and start building immediately.	Click New proposal on the platform and submit it there. Submitting writes revision 1 and dispatches its review in one step.
The decision	A bot reaction plus a DM; the recommendation was for a human maintainer.	A review run on the platform: a verdict per criterion, an overall verdict, and a written final analysis you can read in full.
Self-testing	Mostly Oracle / NOP locally.	You run the static checks AND the rubric review yourself before uploading — the rubric review can be run with free / open-source LLMs.

Submission	Upload, run Oracle/NOP, done.	Upload one immutable Harbor task ZIP against your grant. It enters Stage 1 automatically.
Difficulty	“Agent Trials” as one card.	Stage 2 model rollout, entered only after Stage 1 is accepted. Difficulty and long-horizon gates are decided as verdict predicates.
Fixing issues	Re-upload freely.	Each corrected ZIP becomes a new immutable version (Version N of N) and Stage 1 re-runs against it. Every version is recorded — arrive green.
Final review	QC review.	QC review and then Super QC (a second, expert layer).

2. The TB4 journey at a glance

1. Submit a detailed proposal on the platform: title, category, subcategory, expert time, and the proposal itself in markdown.
2. The Proposal Review Bot judges it and runs duplicate + novelty checks.
3. If accepted and not a duplicate, you receive an ACTIVE grant and it is recorded on that revision. The proposal text is frozen at the grant.
4. Build the task locally — separate verifier structure, no canary lines.
5. Self-test: static checks, rubric review, and Oracle (pass) / NOP (fail).
6. Stage 1 — Static checks. Upload the Harbor task ZIP. G01–G06 and P04 run as one job.
7. Fix and re-upload until Stage 1 reads PASSED STAGE 1.
8. Stage 2 — Model rollout. Rollouts run against a frozen policy and the difficulty gates are decided: PASS or FAIL.



One row per task. The S0 / S1 / S2 marks show, at a glance, how far each task has travelled; the Blocking reason column names the exact rule that stopped a task.

3. Step 1 — Write and submit your task proposal (on the platform)

TB4 begins on the platform, not in your editor. Click New proposal, fill the form, and submit. This gate is cheap and fast on purpose — it stops you sinking hours into a task that turns out to be unverifiable, too easy, or not worth building.

3.1 What the reviewer grades

Every criterion is judged separately, then once overall. A task is only accepted if it is correct, solvable, verifiable, well-specified, and hard for an interesting reason.

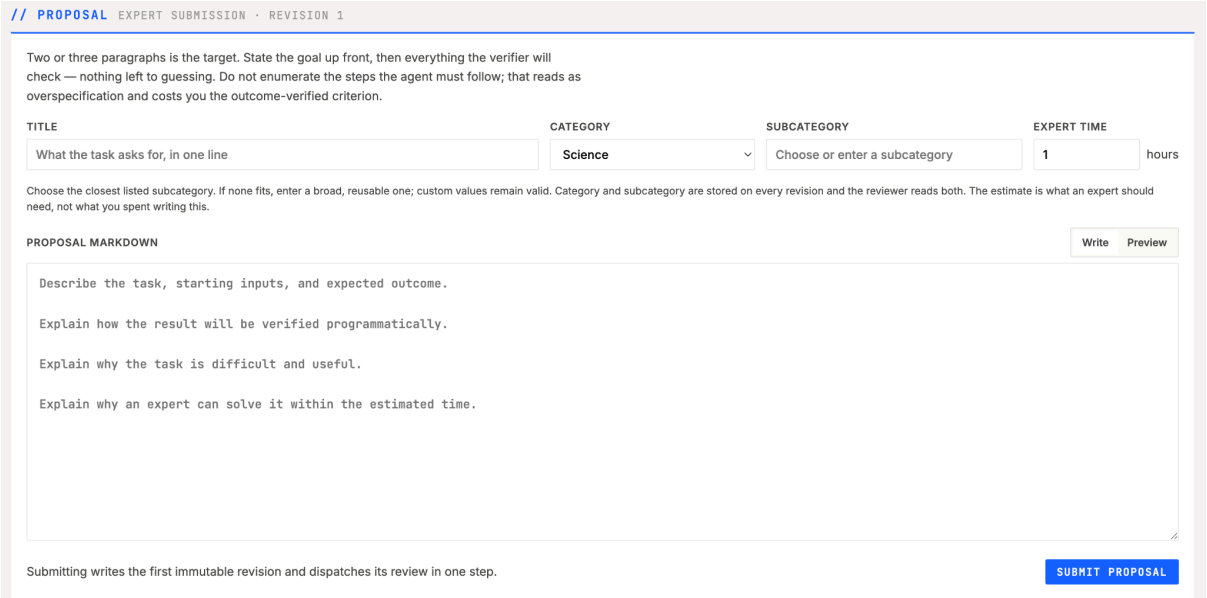
Criterion	What it means
Verifiable	A program separates correct from incorrect, and any other reasonable verifier would agree. Subjective grading is rejected outright.
Well-specified	Everything the verifier checks is stated. Two readers should independently write verifiers that agree.
Solvable	Give a solution or a convincing argument one exists — implementable in a few hours by an expert who knows the answer.
Difficult	Years of domain expertise, not volume of work. Anything an average undergraduate could finish in a few days is too easy.
Interesting	Someone could be paid to do this, or a real group wants it solved. Obscure for its own sake does not count.
Outcome-verified	Grade the final state, not the route taken. No prescribed tools and no enumerated step lists.

3.2 The submission form

The form has five fields, and all five matter:

- **Title** — a specific, descriptive task name.
- **Category** — one of Science, Software, ML, Operations, Security, Hardware, Media.
- **Subcategory** — choose the closest listed subcategory. If none fits, enter a broad, reusable one; custom values remain valid. Category and subcategory are stored on every revision and the reviewer reads both.
- **Expert time** — in hours. This is what an expert *should* need to build the task, not what you spent writing the proposal.
- **Proposal markdown** — the proposal itself. There is a Write / Preview toggle; check the preview before submitting.

Submitting writes the first immutable revision and dispatches its review in one step. There is nothing else to do afterwards but wait for the verdict.



The submission form. The criteria the reviewer will grade you against are shown next to the form, so there is no excuse for guessing at the bar.

3.3 Write the proposal in detail

A strong proposal lets the bot (and later a human) understand the task in a couple of minutes and judge whether it clears the TB4 bar. Describe each of the following clearly:

- Summary & objective — the core scenario in plain language, and the exact deliverable: the precise output file path and type the agent must produce (e.g. /app/report.json).
- Environment & inputs — what the agent starts with (binaries, a repo, datasets, log files, an API) and any helper tools or pinned engines pre-installed in the container.
- Why it is hard — the “anti-AI” trap: the naive approach an LLM tries first, and exactly why that lazy approach fails. Spell out the real source of difficulty (e.g. cross-source correlation, adversarial noise, cascading errors).
- Verification & grading — how correctness is checked deterministically (no LLM judges), how many checks, and how you stop the agent hardcoding the answer (e.g. fresh/frozen fixtures, exact-value checks).
- Real-world relevance & feasibility — who actually does this work and why it matters, and whether the data/environment already exists or must be built.
- Why it is distinct — a sentence on how it differs from existing TB tasks and from obvious public problems (this directly helps you pass the novelty and duplicate gates).

Tip: the more concretely you describe the deliverable, the inputs, and the difficulty, the more reliably the bot can judge it — vague proposals tend to score “Uncertain” or get rejected. Important-Keep it under 2000 characters.

3.4 The verdict

The review run returns a verdict per criterion and one overall verdict. The values you may see are:

- **Accept**
- **Uncertain**

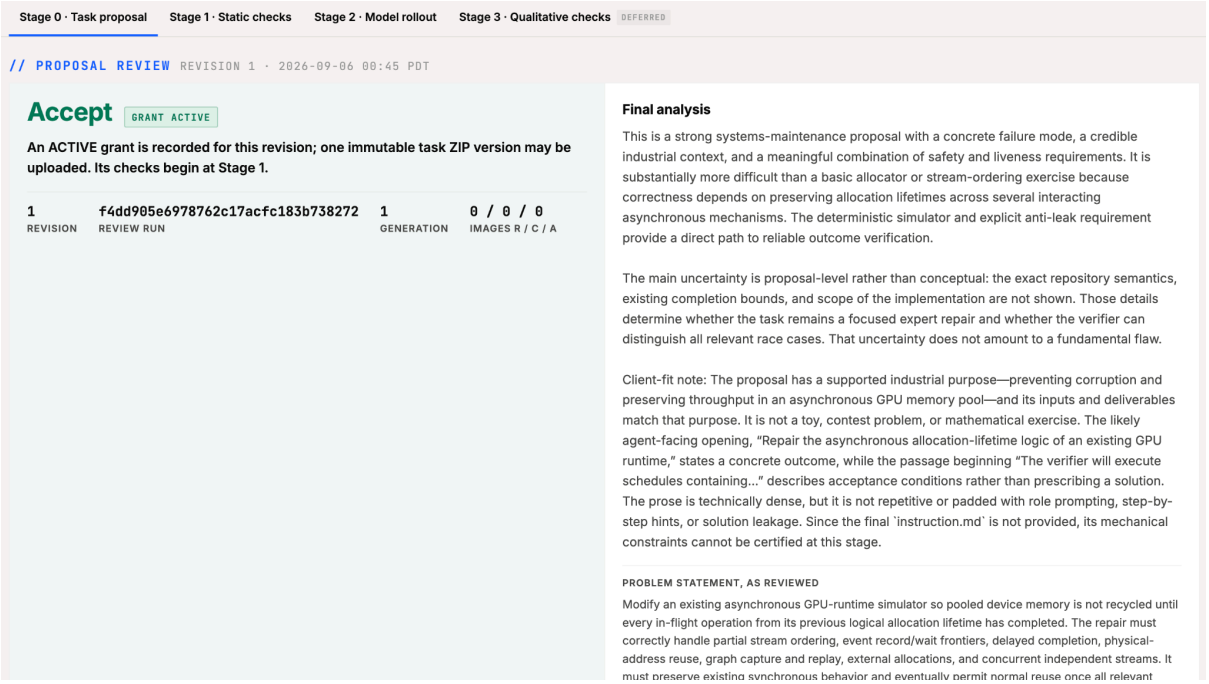
- **Reject**

Alongside the verdict you get a **Final analysis** — a written assessment of the proposal's merits and its weaknesses — and a **Problem statement, as reviewed**, which is the reviewer's own restatement of your task. Read that restatement carefully: if it does not match what you meant, your proposal is not yet well-specified, whatever the verdict says.

3.5 The grant — what actually lets you upload

When a proposal is accepted, an **ACTIVE grant** is recorded for that revision. In practice:

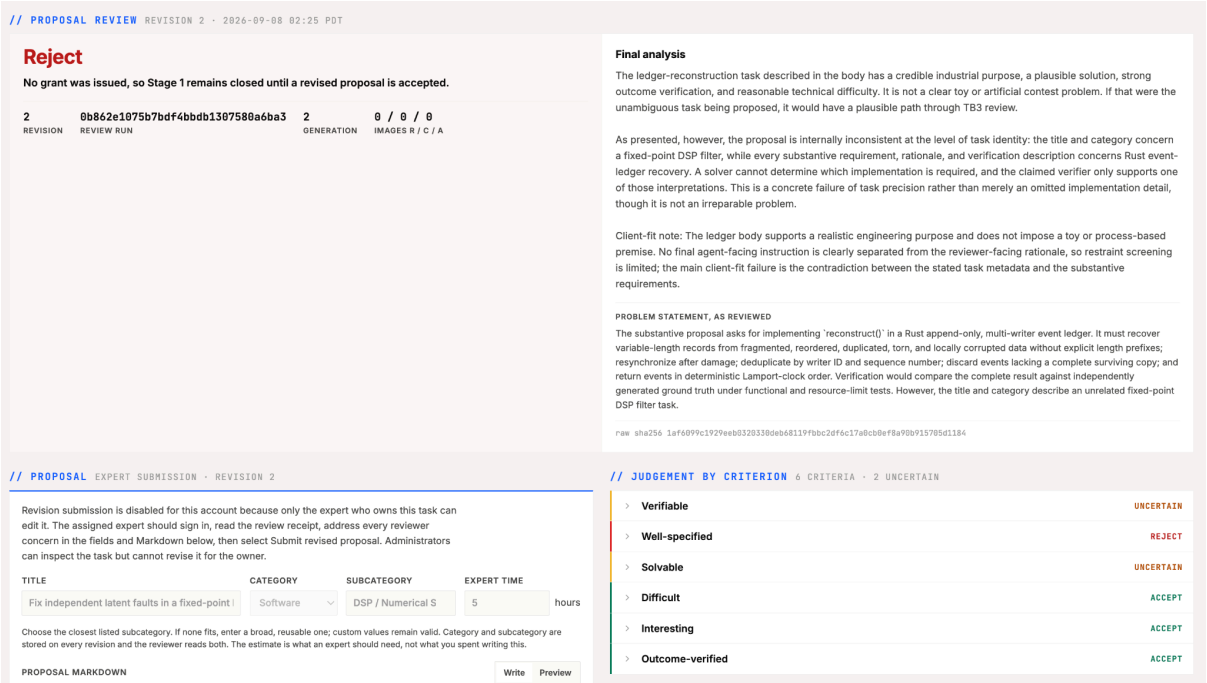
- The grant is your permission to upload. One immutable task ZIP version may be uploaded against it, and its checks begin at Stage 1.
- The proposal text is **frozen at the grant**. The revision you were granted on is the one your task will be judged against. To change the proposal materially, create a new task.
- An administrator can **suspend** or **revoke** a grant. Either closes Stage 1 upload and is recorded with their reason.



An accepted proposal. The ACTIVE grant on the revision is what opens Stage 1 upload; the per-criterion strip shows exactly where the proposal was strong and where it was only just adequate.

3.6 IMPORTANT: If the proposal is rejected or uncertain

The task's status becomes **Proposal rejected — revise proposal**. Read the final analysis, address the specific criterion that failed, and submit a revised proposal. Revisions are numbered and immutable; nothing you already submitted is edited in place.



4. Step 2 — Build the task locally (TB4 structure)

With your idea approved and your canary in hand, you build the task the TB4 way. The biggest structural change from TB1/TB2 is the separate verifier: your tests live in their own container that runs after the agent's container is torn down, so the agent can never read or tamper with your tests, solution, or ground-truth data.

4.1 A TB4 task is more than five files

TB1/TB2 examples were often a handful of files. A real TB4 task usually has more — input data fixtures, a multi-file solution, and a real verifier image. That is expected and encouraged. The layout below is a real TB4 task (a DFIR log-reconstruction task) and shows the kinds of extra files you will commonly add:

```
tasks/<task-name>/
  instruction.md            # Task statement. Canary on line 1. Absolute paths.
No hints.
  task.toml                 # Metadata + resources + timeouts + artifacts +
verifier mode
  environment/
    Dockerfile              # AGENT image. Clean. No tests/, no solution/.
    data/                   # Input fixtures the task ships to the agent:
      access.log  auth.log  dns.log  firewall.log  vpn.log
      users.csv  stage_weights.json  exempt_compromise.txt
      reconstruction_spec.md   # normative output spec the agent must follow
  solution/
    solve.sh                # Oracle entry point (thin launcher)
    solve.py                # The real reference logic (kept as its own file)
  tests/
    Dockerfile              # VERIFIER image. Owns /tests/*. Pre-installs pytest
etc.
```

```
    test.sh                 # Verifier entry point. No runtime installs.
    test_outputs.py         # Strict, behaviour-based checks (pytest, CTRF
output)
```

Two things worth calling out from that example: the solution is split into solve.sh (a thin launcher that just runs python3 /solution/solve.py) and solve.py (the actual logic) — this is the preferred pattern for anything longer than ~20 lines. And the input data lives under environment/data/ and is copied into the agent image by environment/Dockerfile.

4.2 Task Construction Requirements

- **No AI-generated fingerprints.** Task descriptions must carry no obvious traces of LLM synthesis and must pass human review. A description written in tidy markdown structure (e.g. ## Headings) and excessive length is taken as "pure LLM synthesis without human review" and judged non-compliant.
- **Concise, restrained instructions.** The instruction must be concise: word count via content.split() must not exceed 1500. For reference, TB4 averages 267 words, with a maximum of 1406.
- **Precise instructions, no steering.** The instruction must contain no leading or suggestive phrasing (e.g. "please think step by step," "you are an expert in xxx"). Where paths appear, they must be absolute (e.g. /app/solve.py); relative paths (e.g. ./data.csv) are prohibited.
- **Industrial-grade, realistic scenarios.** Tasks must carry real-world significance and a genuine engineering context. Demos, toy projects, and competitive-programming problems (e.g. LeetCode, math olympiad problems) are strictly prohibited.
- **Not exam-style.** All quiz- and contest-style problems — that is, any task built from the premise of "invent a hard problem to test the model" — fall outside the scope of this procurement.
- **Benchmarked against TB4.** Difficulty, turn count, category distribution, and instruction style must all align with Terminal-Bench 3/4.

4.3 The rules that catch people

- Separate verifier mode: set [verifier] environment_mode = "separate" in task.toml, declare artifacts = [...] at the top level (this task uses artifacts = ["/app/"]), and have tests/Dockerfile COPY into /tests and pre-create artifact directories with RUN mkdir -p.
- No runtime installs in test.sh: bake every dependency (uv, pytest plugins, scoring libs, oracle binaries, ground-truth data) into tests/Dockerfile. apt-get install / pip install / curl | sh at verify time are flaky and not allowed.
- Clean agent image: environment/Dockerfile must not copy tests/ or solution/, and must not contain test-only dependencies.
- Absolute paths everywhere: /app/data.csv, never data.csv.
- Pin your dependencies: pip/uv installs need ==<version> pins. Do not pin apt packages. Do not call bare nproc.
- Instruction suffix: instruction.md must end with the exact line “You have N seconds to complete this task. Do not cheat by using online solutions or hints specific to this task.”, where N equals [agent].timeout_sec.
- Tests verify behaviour, not strings: execute the agent's output and check it. Avoid grep-for-keywords tests — they are gameable.

- CTRF reports + reward: test.sh runs pytest with CTRF output and writes the final reward (1 pass / 0 fail) to /logs/verifier/reward.txt.
- Metadata is mandatory: fill in author_name, author_email, difficulty_explanation, solution_explanation, verification_explanation, category, tags, and expert_time_estimate_hours. Empty defaults will fail checks.
- Canary in every file (Section 4) — do not forget the solution and test files.

4.4 Test with the Oracle and NOP agents

Agent	Command	Expected result	If it goes wrong
Oracle	harbor run --path . --agent oracle	100% pass, reward = 1.0	Your solution or verifier is broken (bad test.sh logic or a missing dependency in the verifier image).
NOP	harbor run --path . --agent nop	100% fail, reward = 0.0	Your verifier is weak — it passes when nothing was done. Tighten the tests.

4.5 Run the static checks and the rubric review YOURSELF (before uploading)

This is a core TB4 habit and it protects your limited reuploads. The platform will run the static checks and the rubric review on your task — but you can and should run the same checks locally first, so you arrive already green.

- Static checks: no API keys, run in seconds. Configure the same pipeline locally and fix every failure before uploading. Full list in the "Static Checks" tab.
- Rubric review: an LLM grading your task against the implementation rubric. Run it locally with free / open-source LLMs (smaller open models such as Llama- or Kimi-class agents) — it is a self-check signal, so a free model is enough to catch obvious gaps. Full rubric in the "Rubric Review" tab.
- Dry-run the task on free / open-source agents before spending platform trials, to confirm it is solvable, the instruction is clear, and it is not trivially easy.

Why bother when the platform runs them anyway? every issue you catch locally is one you don't spend an upload on — and your uploads are limited (see Section 8).

5. Step 3 — Upload the ZIP and clear Stage 1 (sanity checks)

Once the task is built locally, you are ready to upload. Open your task on the platform and click **Upload Harbor task ZIP** (the button is inactive until an ACTIVE grant exists). The ZIP is immutable once uploaded: the platform binds its SHA-256 and an upload generation, and Stage 1 begins automatically.

Stage 1 is a sequenced job, not a set of cards you trigger one by one. It runs **I01 (**upload snapshot), gates **G01** → **G06,** and then P04 (the publication barrier). A typical clean run finishes in well under a minute, most of it spent in G04 through G06 (environment build, verifier controls, and package-quality checks).

5.1 The Stage 1 gates

ID	Gate	What it checks
I01	Upload snapshot	Binds the immutable upload generation and SHA-256 snapshot of your ZIP before anything reads it.
G01	Archive safety	Exact ZIP inventory, safe extract, and a single candidate root. Catches path traversal, absolute paths, symlinks, encrypted members, CRC mismatches, archive bombs, and multiple/zero task roots.
G02	Harbor + TB4 contract	Harbor layout parses under the pinned Harbor version, all required paths are present and readable, TB4 metadata is valid (category, subcategory, required fields), agent networking is non-public, and verifier isolation is declared.
G03	Static execution safety	No hidden runtime, credentials, or network reach-out. Credential scanning, Compose dependency and provenance scanning, host binds, privileged containers, host networking, devices, packaged-file mutation, Dockerfile references, and verifier fetch patterns.
G04	Environment readiness	Whether Harbor can actually build and start your declared image and Compose environment, and reach bound agent-setup evidence. Unsupported profiles, accelerators, prebuilt images, and timeout envelopes fail here.
G05	Verifier controls	Oracle and NOP run as one Harbor job. Oracle must score 1.0 and exit cleanly; NOP must not score 1.0. Reward must be found, parseable, unambiguous and finite. This is where most real task defects surface.
G06	Package quality	Static package quality - instruction, tests, mapping, and related criteria.
P04	Final publication barrier	Binds this ZIP, its receipt, and its artifacts — sanitises output, closes hashes, persists the result, and publishes the admission — or stops.

Each gate expands to show its **phases** and its individual **rules**. Rules carry a severity, and reading it correctly saves you a lot of guessing:

- **REQUIRED** — raising this rule rejects the task (or, for platform-side rules, marks Stage 1 incomplete).
- **OPTIONAL / ADVISORY** — useful feedback that does not reject the task. A run can read *Stage 1 accepted · 2 warnings* and still be perfectly fine.
- **IF APPLICABLE** — only evaluated when your task actually uses the feature (for example, Compose-specific rules).

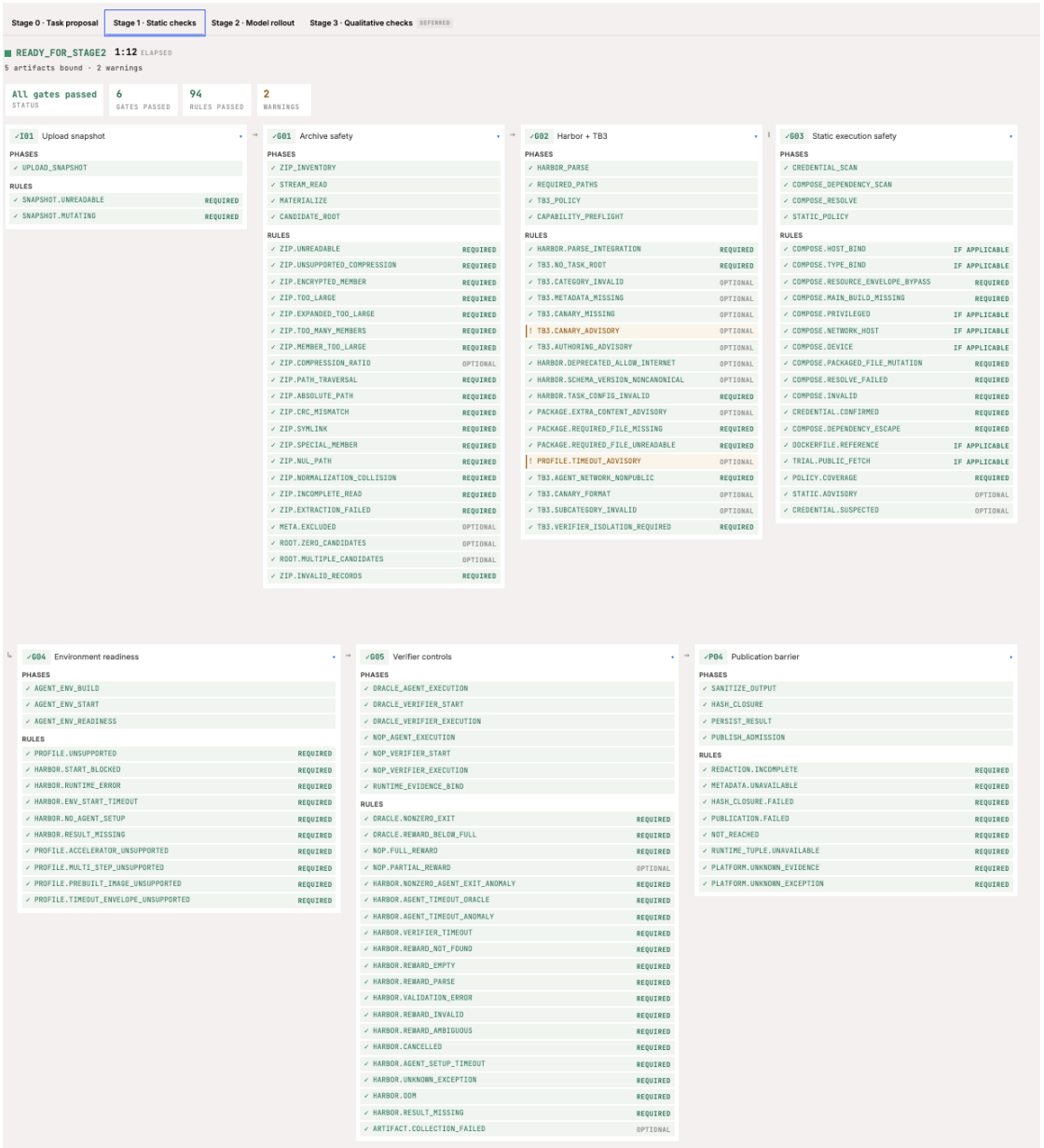
5.2 Reading the Stage 1 outcome

A finished Stage 1 lands in one of three places, and they are not the same thing:

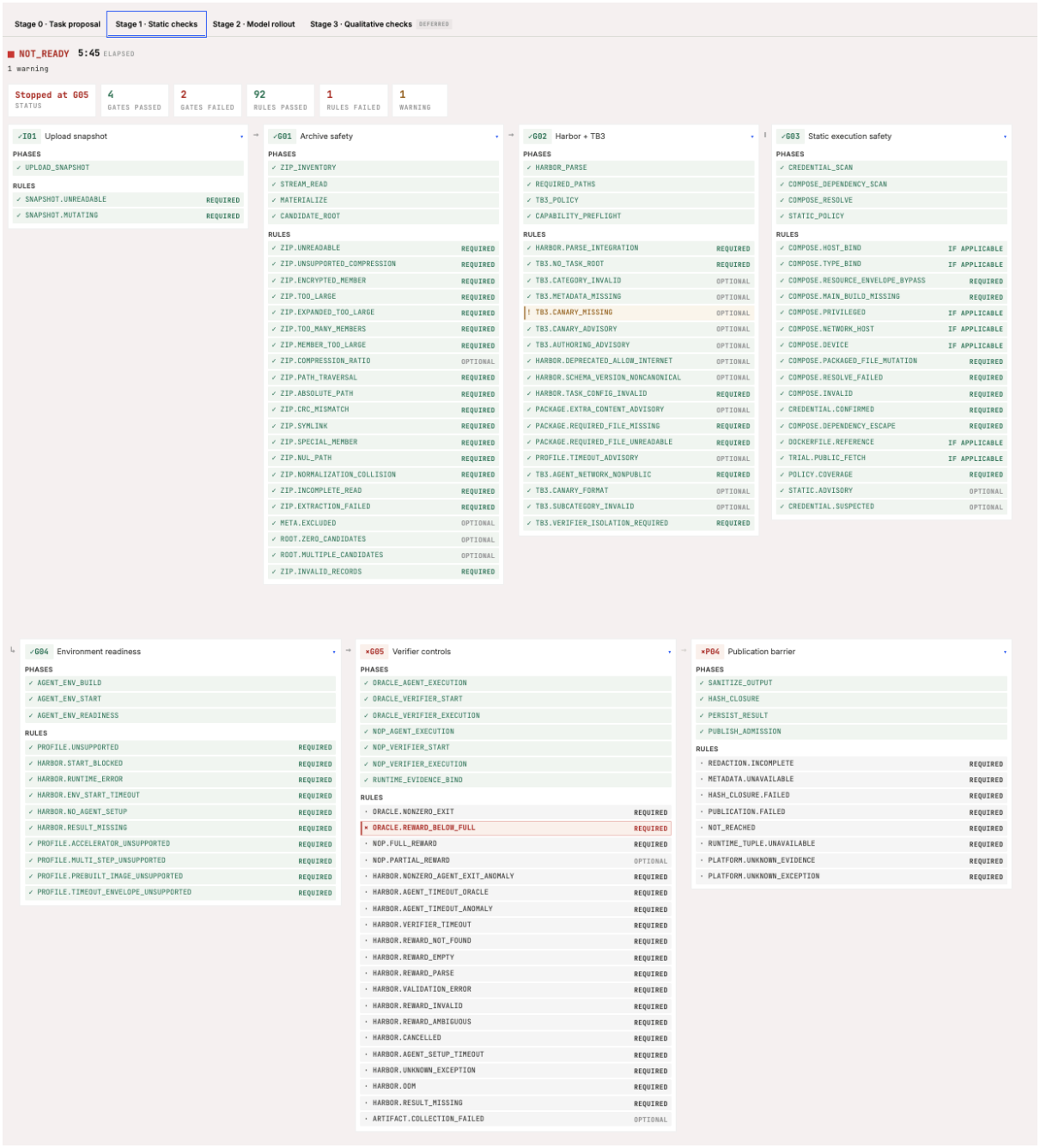
- **Stage 1 accepted / PASSED STAGE 1** (READY_FOR_STAGE2) — all gates passed. Your task is handed off to Stage 2, along with the input ZIP and the config projections it will run from.

- **Stage 1 rejected — upload a corrected ZIP** — a REQUIRED rule found a defect *in your task*. Fix it and upload a new version. The Blocking reason column names the rule, e.g. G05.ORACLE.REWARD_BELOW_FULL.
- **Stage 1 could not complete — retry** — Stage 1 hit something it will not attribute to your task: a Harbor anomaly, an unclassified exception, an OOM or eviction, a cancellation, an unsupported accelerator profile. Examples: G05.HARBOR.UNKNOWN_EXCEPTION, G04.PROFILE.ACCELERATOR_UNSUPPORTED. This is a retry, not a rewrite — check the profile you declared, then re-run.

The distinction is deliberate: the platform fails *closed* rather than inventing a task defect, so "could not complete" is never a verdict on your work.



A clean Stage 1. Six gates passed and every rule green; the summary table underneath gives per-gate duration so you can see where the time went.



A rejected Stage 1. The failing rule states what it enforces, what actually happened, and what to fix — read it before touching anything.

5.3 Fixing and re-uploading

- Open the failing gate and read the rule that raised. It tells you what the rule enforces, what happened on your run, and the fix.
- Use the **durable event log** at the bottom of the Stage 1 tab to see the exact phase ordering and where your run stalled.
- Fix locally, re-run your local static checks and Oracle/NOP, then upload a corrected ZIP. It becomes a new immutable version and Stage 1 re-runs against it.
- Only one thing gets you to Stage 2: every gate green on a single version. Partial credit across versions does not count.

6. Step 4 — Stage 2: model rollout and the difficulty gates

Once Stage 1 is accepted, the task is ready to run the model rollouts. Stage 2 is where difficulty is actually decided, and it is decided by measurement, not by opinion.

Stage 2 moves through a fixed evaluation flow — **QUEUED** → **PREPARING** → **RUNNING** → **COLLECTING** → **CLOSING** → **COMPLETE** — and can sit at *Stage 2 running · waiting on provider* while capacity is allocated. You do not drive it; you read it.

6.1 What Stage 2 runs

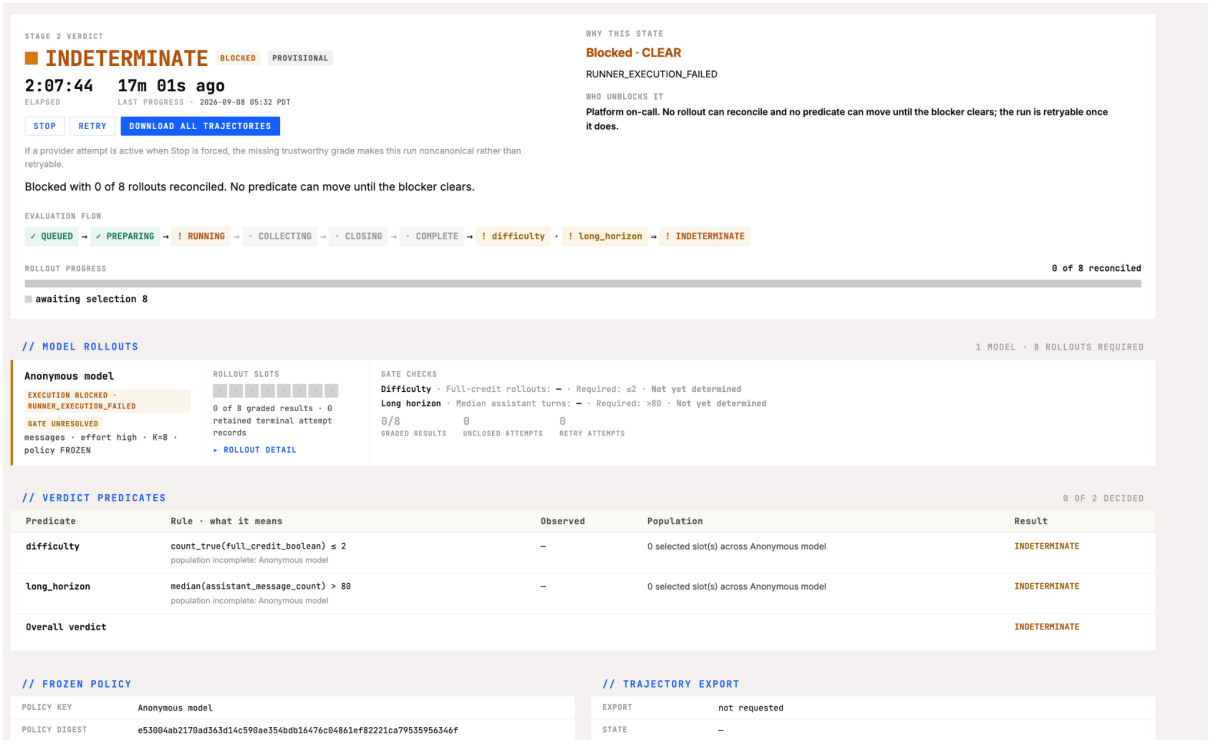
- **8 rollouts** of the task against the evaluation model, under a **frozen policy** (policy key and digest are recorded, along with the capacity and retention profiles).
- Each rollout slot is graded and retained as a terminal attempt record. The tab shows *8 of 8 reconciled*, plus counts of graded results, unclosed attempts, and retry attempts.

6.2 The gates

Stage 2 decides a small number of **verdict predicates** over those rollouts. Each predicate is a stated rule over the selected population, with the observed value shown next to it:

Predicate	Rule	What it means
difficulty	count_true(full_credit_boolean) ≤ 2	At most 2 of the 8 rollouts may earn full credit. More than that and the task is too easy.
long_horizon	median(assistant_message_count) > 80	The task must genuinely take a long agent trajectory. A median of ~48 turns fails this even when difficulty passes.

Every predicate must hold. The **overall verdict** is PASS or FAIL, and the header names the predicate that sank it (for example, *Failed on long_horizon*).



Stage 2 in full. Gate checks show each predicate's observed value against its requirement, so a FAIL always tells you which property of the task was short.

6.3 Reading a Stage 2 FAIL

- **Difficulty not met** (too many full-credit rollouts) — the task is solvable by the model as written. The fix is depth, not obscurity: more cross-source correlation, more state to keep consistent, stricter exactness in the output contract. Do not fix it by hiding information in the instruction.
- **Long horizon not met** (median turns too low) — the task is hard but short. It needs more genuine work in sequence: more components that must all hold together, more recovery and boundary conditions the agent has to discover and handle. Padding with busywork does not survive Stage 3, and it will read as busywork.
- Both predicates are computed over the same 8 rollouts, so a change aimed at one usually moves the other. Re-check both before you resubmit.

7. Before you upload — quick checklist

- Proposal submitted on the platform with title, category, subcategory and expert time filled in, and an **ACTIVE grant** recorded on the revision.
- Task built to the TB4 structure with **separate verifier mode** declared and artifacts set.
- **No canary lines anywhere.** Nothing in your files, nothing to paste at upload.
- Absolute paths only; dependencies pinned; no runtime installs in test.sh; verifier tooling baked into tests/Dockerfile.
- All required metadata fields filled — no empty defaults.
- instruction.md under 1500 words, no steering language, correct timeout suffix line.
- Oracle scores 1.0 locally; NOP scores 0.0 locally.
- Local static checks and rubric review both clean.

- Dry-run on a free/open-source agent suggests it is solvable but not easy, and that it takes a long trajectory.

TB4 — Static Checks Reference

Companion to the TB4 Proposal & Submission Guide

Static checks are fast, deterministic, rule-based checks that need no API keys and no LLM. They catch common mechanical mistakes — a missing canary, a leaked solution, an unpinned package, a relative path. They do NOT judge whether your task is good or hard (that is the rubric review); they only confirm it is structurally well-formed. Because they are cheap and run in seconds, run them locally over your task before every upload, so you never waste a reupload on a mechanical slip. Below is every check and exactly what it enforces.

Static checks

Static check	What it enforces
check-dockerfile-references	The agent Dockerfile does not copy solution/solve.sh, tests/test.sh, or tests/test_*.py into the image. The agent must never see the tests or solution.
check-dockerfile-sanity	Dockerfile best practice: apt packages are NOT pinned to versions, and apt install is preceded by apt-get update and followed by rm -rf /var/lib/apt/lists/* cleanup.
check-dockerfile-platform	No Dockerfile hardcodes a CPU architecture via FROM --platform=..., so the task runs on any backend (x86-64 or arm64) without slow emulation.
check-task-absolute-path	Instructions use absolute paths (e.g. /app/data.csv), never relative ones (data.csv).
check-test-file-references	Every output file referenced by both the tests and the solution is documented in instruction.md or task.toml.
check-test-sh-sanity	test.sh isolates Python using uv venv (or task.toml / instruction.md explicitly documents system-wide Python usage).
check-task-fields	All required task.toml metadata is present: author_name, author_email, difficulty_explanation, solution_explanation, verification_explanation, category, tags, expert_time_estimate_hours.

check-task-timeout	[agent] and [verifier] timeout_sec are capped at 18000s (5 hours), leaving headroom under the CI runner's 6-hour job limit.
check-instruction-suffix	instruction.md ends with the exact line “You have N seconds to complete this task. Do not cheat by using online solutions or hints specific to this task.”, where N equals [agent].timeout_sec, as its own paragraph with a single trailing newline.
check-gpu-types	Any gpu_types values are canonical Modal GPU strings (T4, L4, A10, L40S, A100-40GB, A100-80GB, H100, H200, B200, or any).
check-task-slug	The task folder name is at most 3 hyphen-separated tokens, lowercase kebab-case.
check-allow-internet	task.toml does not set [environment].allow_internet = false — tasks run with open internet by default.
check-separate-verifier	The task is fully configured for separate verifier mode: environment_mode = "separate", artifacts declared at top level, tests/Dockerfile exists and COPYs into /tests, and artifact parent directories are pre-created with RUN mkdir -p.
check-trial-network-fetch	tests/test.sh does not fetch external resources at verify time (no curl|sh / wget|sh installers, no external curl/wget, no git clone of external URLs) — a known reward-hacking surface.
check-pip-pinning	Every pip / uv pip / uvx / uv tool install pins an explicit ==<version>. Manifest (-r requirements.txt), local (-e ., .), pip self-upgrade, and VCS URLs with an @ref are allowed.
check-nproc	No bare nproc (it returns the host CPU count inside a container, causing over-parallel OOM/timeouts). Use the configured [environment].cpus or a fixed value instead.
check-ai-prompt-sanitization	AI prompts do not leak the task solution or hidden data to the model.

Run these locally first. None of these need an API key. Fixing a static-check failure on your own machine costs nothing; fixing it after upload costs one of your three reuploads.

TB4 — Rubric Review Reference

Companion to the TB4 Proposal & Submission Guide

Where static checks ask “is the task well-formed?”, the rubric review asks “is the task good?”. It is an LLM-based evaluation: a model reads your instruction, environment, solution, and tests and grades the task against a set of quality criteria, each as PASS / FAIL with a justification. The platform runs this as a Quality Dimension, but you can run it yourself locally with a free / open-source LLM before uploading — it is a self-check signal, so a smaller open model is enough to surface obvious gaps. The criteria below are grouped by theme; aim for every one to PASS.

Group A — Verifiability & correctness

Criterion	What the reviewer looks for
Deterministic verification	Verification is fully deterministic — no LLM-as-judge, no fuzzy matching that could pass wrong answers or fail right ones.
Outcome-based grading	Tests check the agent's final output / artifact, not the specific steps or method used to get there.
Frozen ground truth	Expected values come from frozen, in-image fixtures so results are reproducible and not time/network dependent.
Complete coverage	Tests cover the full required output (all fields, edge cases), not just a happy-path subset.
Correct expected values	The expected answers the tests assert are themselves correct (independently derived / verified).
Strict failure on no-op	A do-nothing or partial attempt fails — the verifier is not lenient.

Group B — Solvability

Criterion	What the reviewer looks for
Reference solution exists	A correct reference solution is provided and passes all tests (Oracle = 1.0).
Solvable from instruction alone	Everything needed to solve the task is in the instruction and environment — no external/hidden knowledge or missing inputs.

Environment is self-contained	All required tools, data, and dependencies are present in the container; the task does not rely on flaky external resources.
Achievable in the time budget	A capable agent can plausibly finish within [agent].timeout_sec.
No ambiguity in the goal	The success condition is unambiguous — there is one clearly correct interpretation of “done”.
Output schema fully specified	The exact output path, format, schema, ordering, and formatting are spelled out so a correct attempt isn't failed on a guessable detail.

Group C — Difficulty & anti-cheat

Criterion	What the reviewer looks for
Genuinely difficult	The task is hard for a strong agent — not solvable by a trivial one-liner or obvious lookup.
Non-trivial (NOP fails)	Doing nothing, or an obvious naive approach, does not pass.
Difficulty is legitimate	Hardness comes from real reasoning/engineering, not from obscure trivia, trick wording, or guess-the-format.
Resistant to reward hacking	The agent cannot game the verifier (e.g. hardcoding expected values, writing to reward files, reading hidden tests/solution).
Answer not leaked	The instruction, environment, and data do not reveal the expected answer or the test internals.
No shortcut via internet	The answer cannot simply be looked up online or copied from a public source.

Group D — Instruction clarity & quality

Criterion	What the reviewer looks for
Clear and well-written	The instruction is unambiguous, well-structured, and free of contradictions.
No hidden hints or leakage	It does not hint at the solution method or leak verifier details.
Absolute paths & concrete I/O	All file paths are absolute and every required input/output is named explicitly.
Required suffix present	Ends with the exact timeout / anti-cheat line matching the agent timeout.

Self-consistent with tests	What the instruction asks for is exactly what the tests check — no drift between statement and grading.
Appropriately scoped	The task is a focused, single objective — not a sprawling bundle of unrelated sub-tasks.

Group E — Structure, environment & realism

Criterion	What the reviewer looks for
Separate verifier configured	Tests run in their own container after the agent; the agent image is clean (no tests/solution).
Dependencies pinned & baked	All deps are version-pinned and pre-installed in the right image; no runtime installs at verify time.
Sensible resources	CPU / memory / storage / timeouts are appropriate — not wastefully large, not too tight to solve.
Reproducible build	The images build cleanly and deterministically on a fresh machine.
Real-world relevance	The task reflects a realistic, valuable scenario someone would actually do — not a contrived puzzle.
Correct & complete metadata	task.toml metadata (difficulty, solution, verification explanations, category, tags, time estimate) is filled in and accurate.

How to use this locally: run the rubric review with a free / open-source LLM over your task before uploading. Read each FAIL with its justification, fix the underlying issue, and re-run. Arriving on the platform already passing the rubric protects your 3 reuploads.

1. Limits you should know

Resource	Limit
Reuploads (new versions) per task	7
Total submissions	1 (account-wide)
Hourly evaluation limit	2 unique versions
Daily evaluation limit	5 unique versions
Task creation limit	2 tasks

This is why you self-test: with only 3 reuploads per task, every static-check, rubric, and Oracle/NOP failure you catch locally is a reupload you do not waste on the platform.

Locked at creation: once a task is created you cannot change its category or programming language — choose carefully.

2. Troubleshooting & common pitfalls

Symptom	Likely cause & fix
Proposal blocked (red) despite a good rubric verdict	Duplicate detected — similarity above threshold. The idea exists already; rework it into something genuinely distinct.
Proposal rejected for novelty	It is semantically the same as an existing task. Surface-level changes are not enough.
Never received the canary DM	DMs from server members are disabled. Enable them in Discord privacy settings, then ask a moderator to re-send.
Proposal Match FAILED	canary_string missing/mistyped, canary not in all files, or the built task diverged from the approved proposal.
Agent Trials / Cheat Trial stay blocked	Proposal Match has not passed yet. Clear it first.
Harbor Analyze blocked	Agent Trials must complete first.
Static Checks failed	Run them locally and fix every flag (see the Static Checks Reference).
Rubric Review flagged criteria	Run the rubric review locally with a free LLM and address the FAILs (see the Rubric Review Reference).
Oracle fails	Broken verifier logic or a dependency missing from tests/Dockerfile.
NOP passes	Verifier too lenient — it accepts work never done. Make the tests strict.

Ran out of reuploads	You hit 3 versions for that task. Self-test locally next time; you may need a new task.
“File hash already exists” on upload	You re-uploaded an identical ZIP. Make a real change first.

3. Final pre-submission checklist

- Proposal posted, accepted, and NOT flagged as duplicate or non-novel.
- Discord DMs from server members enabled; canary received.
- Canary present (inside a comment) in EVERY text file: instruction.md, task.toml, environment/Dockerfile, all of solution/ and all of tests/.
- Canary entered as the canary_string field at upload.
- Separate verifier configured (environment_mode = "separate", artifacts declared, tests/Dockerfile owns /tests).
- Agent image clean; no tests/ or solution/ copied in; no test-only deps.
- All dependencies pinned; no runtime installs in test.sh; no bare nproc.
- Absolute paths everywhere; instruction ends with the exact timeout/anti-cheat line.
- Tests verify behaviour (not string matching) and emit CTRF + reward.txt.
- All required task.toml metadata filled in (no empty defaults).
- Static checks run locally and all pass (Static Checks Reference).
- Rubric review run locally with a free LLM; all criteria addressed (Rubric Review Reference).
- Oracle passes 100% and NOP fails 100%.
- All 9 Quality Dimensions green on the platform.

Categories to collect

We follow the official taxonomy of Terminal Bench 3. Delivered tasks must broadly cover the categories and tags used across TB4. No single category may exceed 20% of the delivered batch.

Science, Operations, and Media (deprioritized categories) are included in this round as thin presence slices (1 task each subcategory) so the bench covers more of the official taxonomy without displacing the Software / ML / Security / Hardware focus.
Do not concentrate the batch in Software & Science. Operations, Hardware, Security, Media, and ML tags are in-scope on equal footing, subject to the 20% cap.

Category	Sub-category	Description
Software	Systems	Low-level and infrastructure work — storage engines, concurrency, crash recovery, schedulers, sysadmin-grade debugging. Example: fix write-ahead-log recovery so a storage engine survives crash-and-replay with correct ordering semantics. Largest single subcategory in the release.
	Algorithms	Correctness- and complexity-driven problems where the difficulty is in the logic, not the environment. Includes performance work, since the guide treats speed and efficiency as first-class — correctness alone isn't enough. github
	Frontend	Browser-facing application work: UI state, validation, client/server contracts. Example: a broken React/TypeScript lead-intake app whose submission pipeline must stay consistent across validation, ledger reconciliation, and CRM export.
	Data engineering	Streaming and batch pipelines. Example: debugging five interacting failures in a stream-processing engine's session-window GC, merge, and watermark logic, each guarded by a misleading comment.
	Databases	Query engines, schema and migration work, transactional semantics, performance tuning. Databases are named as one of the richer environments agents must explore, github so these often run multi-container.
ML	Inference	Serving-side work — latency, throughput, batching, deployment config. Framed as LLM inference optimization: tuning serving for latency and throughput. github Largest ML subcategory.
	Evaluation	Building and repairing measurement machinery: metrics, statistical tests, monitoring. Example: repairing a drift monitor whose statistical tests and alerting layer break in ways that mimic real production noise.
	Training	Post-training and optimization loops — SFT and RL of small language models. github Bounded by the hardware limit, since GPU tasks get a single H100. github

	Kernels	GPU programming: writing and optimizing CUDA or Triton kernels. Enabled specifically by the addition of GPU resource containers. github Smallest ML bucket.
Security	Cryptography	Breaking or correctly implementing cryptographic constructions — weak primitives, protocol flaws, key recovery. Verification is unusually clean: you recovered the plaintext or you didn't. github
	Forensics	Reconstructing events from artifacts — packet captures, logs, disk images, memory dumps. Evidence-chaining over long horizons, matching the preference for iterative tasks solved through trial and error, not read-and-solve. github
	AppSec	Application-layer vulnerability discovery and remediation — injection, auth flaws, unsafe deserialization, dependency risk. Smallest security bucket and the closest in shape to ordinary software engineering.
Hardware	CAD	Programmatic mechanical and geometric modelling — parametric solids, tolerances, exported artifacts checked against geometric ground truth. The larger of the two hardware buckets.
	RTL	Register-transfer-level digital design: Verilog/VHDL, testbenches, simulation, synthesis-aware correctness. A single-task slice, but proof the benchmark reaches work that isn't software engineering at all.
Science	Biology	genomics, proteomics, structural/computational biology, bioinformatics.
	Chemistry	molecular structure, spectra, crystallography, reaction/property analysis.
	Physics	simulation and numerical modeling of physical systems, including electromagnetic/FDTD device simulation and inverse design.
	Earth	earth, climate, hydrology, and environmental modeling
	Robotics	dynamics, trajectory optimization, control.
	Math	formalized mathematics and theorem proving (Coq/Lean/Isabelle), formal verification.
	Linguistics	computational and historical linguistics.
Operations	Finance	quantitative finance, risk, regulatory capital.
	Logistics	dispatch, routing, fleet/flight planning, transportation.
	Supply chain	procurement, production planning, manufacturing, ERP.

	Claims	insurance/utility claims, billing rules, adjudication.
	Compliance	regulatory reporting and compliance workflows.
	Marketing	ads, CTR, marketing analytics.
	Music	music theory, audio transcription and processing.
Media	Design	visual/layout design and reconstruction

* Note: The distributions are just rough numbers out of 66 total tasks.

Acceptance Criteria — Task Submissions

Taxonomy & Volume

Please see the Taxonomies tab of this doc.

Model-Breaking Difficulty and Solvability Gates

We use a 3-gates process to access the model-breaking level of a submitted task:

1. **Difficulty**: Roll out with an anonymous model, timeout limit 8 hrs, 8 rollouts, pass rate ≤ 0.6.
2. **Long horizon Gate:** We require **a minimum of 35 turns of all passing runs in the Anonymous model**.
3. Tasks that fail both difficulty columns, fail solvability, or fail both turn-count columns are rejected.

Note: Anonymous model - This model is close to GPT-6-Sol-xhigh performance

Difficulty and horizon metrics

Dimension	Latest Frontier Model
Pass rate (8 rollouts, timeout limit 8 hrs)	≤ 0.6 i.e 4/8
Turns (role=assistant)	Anonymous > 35 turns

Review Pipeline Compliance

Each task must pass the full automated + human review pipeline defined in `TASK_REVIEW_AUTOMATION.md` before it counts as accepted:

- Static checks — path validation, Dockerfile sanity, canary strings, metadata completeness, test-reference integrity
- Implementation rubric review — automated `harbor check` scoring against the task-implementation rubric
- Docker / Oracle / No-op validation — environment builds cleanly, the reference solution (oracle) passes, and a no-op run fails
- Maintainer review — sign-off once all automated gates are green
- Agent & cheat trials — live agent trials plus adversarial cheat trials specifically probing for reward hacking

- Hacker–fixer loop — iterative hardening until an adversarial agent can no longer exploit the verifier

A task is considered accepted only once it clears all four criteria and is merged following this pipeline — not upon submission or upon passing static checks alone.